

Universidade Federal de Santa Catarina – UFSC
Centro Tecnológico – CTC
Departamento de Informática e Estatística – INE

INE5414 - Redes de Computadores I

Trabalho Prático 3

Discente:
Ana Luiza Sales Gobbi (24105453)

Florianópolis, 15 de novembro de 2025

Resumo

Durante a realização deste trabalho, utilizei os conhecimentos adquiridos em aula para avaliar e compreender os conceitos relacionados às ferramentas de redes. Fui feito uso do Wireshark para capturar pacotes e analisar os protocolos ARP, UDP, TCP e HTTP

Sumário

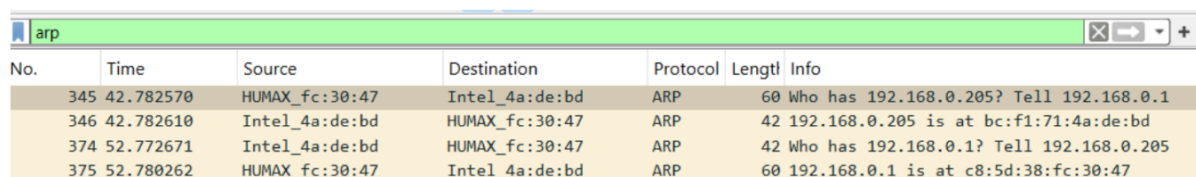
1 Introdução.....	4
2 ARP.....	4
3 TCP.....	4
3.1 Criação de conexão TCP.....	7
3.2 Encerramento da conexão TCP.....	8
3.3 Sobre a captura de pacotes.....	8
4 HTTPS.....	9
5 UDP.....	12
6 Conclusão.....	12
7 Bibliografia.....	12

1 Introdução

Este trabalho tem como objetivo aplicar, de forma prática, os conceitos de gerência de redes por meio do uso de uma ferramenta de monitoramento. A proposta consiste em analisar o tráfego de uma rede local, com ênfase na captura e interpretação de pacotes dos protocolos TCP e HTTP, utilizando conhecimentos adquiridos ao longo da disciplina e em pesquisas complementares. Para a execução das atividades, optei pelo uso do Wireshark, uma ferramenta gratuita e robusta que permite monitorar, filtrar e identificar pacotes de dados na rede em tempo real. A escolha se deu tanto pela eficiência dos seus recursos de filtragem quanto pela familiaridade adquirida com o software em trabalhos anteriores da disciplina. Além disso, o Wireshark oferece a possibilidade de salvar as capturas em arquivos próprios, o que facilita o armazenamento, a análise posterior e o compartilhamento dos dados monitorados em diferentes ambientes de trabalho.

2 ARP

O ARP (Address Resolution Protocol) mapeia endereços da camada de Internet (IPv4 ou IPv6) em endereços físicos MAC, sendo fundamental para o funcionamento de redes locais. Depois de um monitoramento contínuo de 1 minuto, salvei o arquivo de captura e o analisei usando os filtros de interesse. Abaixo, segue uma imagem de uma seção da captura filtrada com por ARP.



No.	Time	Source	Destination	Protocol	Length	Info
345	42.782570	HUMAN_fc:30:47	Intel_4a:de:bd	ARP	60	Who has 192.168.0.205? Tell 192.168.0.1
346	42.782610	Intel_4a:de:bd	HUMAN_fc:30:47	ARP	42	192.168.0.205 is at bc:f1:71:4a:de:bd
374	52.772671	Intel_4a:de:bd	HUMAN_fc:30:47	ARP	42	Who has 192.168.0.1? Tell 192.168.0.205
375	52.780262	HUMAN_fc:30:47	Intel_4a:de:bd	ARP	60	192.168.0.1 is at c8:5d:38:fc:30:47

Figura 1: Captura de pacotes com filtro ARP

A captura de pacotes ARP (Address Resolution Protocol) demonstra o mapeamento de endereços lógicos (IP) para endereços físicos (MAC) na sua rede local.

Troca Principal (Mapeamento do Roteador)

- Dispositivo Solicitante: O dispositivo com o endereço MAC Intel_4a:de:bd envia uma solicitação ARP (pacote 374): Who has 192.168.0.1? Tell 192.168.0.205. Este solicitante (provavelmente seu computador, IP 192.168.0.205) está tentando descobrir o endereço MAC do seu roteador (IP 192.168.0.1).
- Dispositivo Respondente: O dispositivo com o endereço MAC HUMAN_fc:30:47 (que é o roteador) responde (pacote 375): 192.168.0.1 is at c8:5d:38:fc:30:47.
- Resultado: A resposta fornece o endereço MAC (c8:5d:38:fc:30:47) associado ao IP do roteador, permitindo que a comunicação na Camada 2 seja estabelecida.

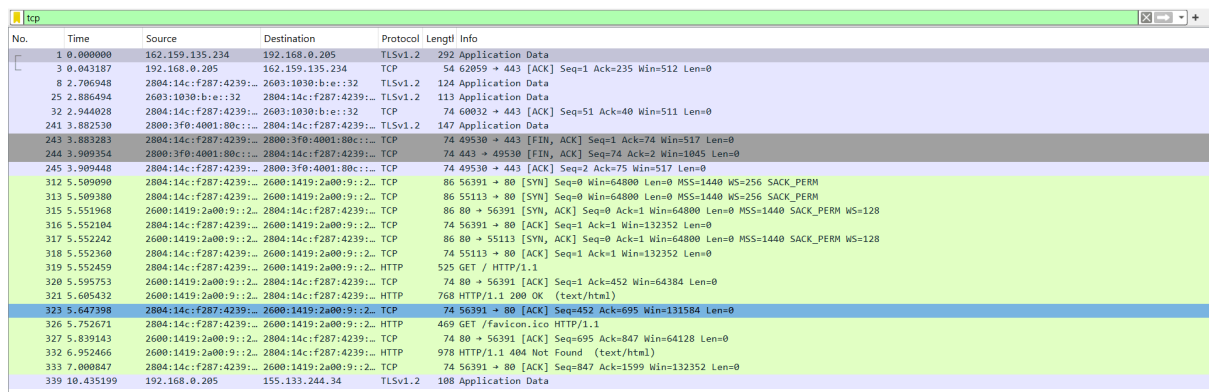
Troca Secundária (Mapeamento do Computador)

- Dispositivo Solicitante: O dispositivo HUMAX_fc:30:47 (o roteador) envia uma solicitação ARP (pacote 345): Who has 192.168.0.205? Tell 192.168.0.1. O roteador está tentando obter o endereço MAC do seu computador.
- Dispositivo Respondente: O dispositivo Intel_4a:de:bd (seu computador) responde (pacote 346): 192.168.0.205 is at bc:f1:71:4a:de:bd.
- Resultado: O roteador obtém o endereço MAC (bc:f1:71:4a:de:bd) do seu computador.

Esta troca de requisição e resposta (ARP Request → ARP Reply) é fundamental para o funcionamento da rede local, permitindo que os dispositivos se comuniquem diretamente após obterem o mapeamento necessário.

3 TCP

O TCP é um protocolo orientado a conexão que garante a entrega de dados de forma confiável e ordenada. A principal característica é o uso de mecanismos de controle para estabelecer, gerenciar e encerrar a comunicação de forma segura, o que é demonstrado no ciclo de vida da conexão.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	162.159.135.234	192.168.0.205	TLSv1.2	292	Application Data
3	0.043187	192.168.0.205	162.159.135.234	TCP	54	62059 → 443 [ACK] Seq=1 Ack=235 Win=512 Len=0
8	2.706948	2804:14c:f287:4239::	2603:1030:b:e::32	TLSv1.2	124	Application Data
25	2.886494	2603:1030:b:e::32	2804:14c:f287:4239::	TLSv1.2	113	Application Data
32	2.944028	2804:14c:f287:4239::	2603:1030:b:e::32	TCP	74	60032 → 443 [ACK] Seq=51 Ack=40 Win=511 Len=0
241	3.862530	2800:3f0:4001:80c::	2804:14c:f287:4239::	TLSv1.2	147	Application Data
243	3.863283	2804:14c:f287:4239::	2800:3f0:4001:80c::	TCP	74	49530 → 443 [FIN, ACK] Seq=1 Ack=74 Win=517 Len=0
244	3.999354	2800:3f0:4001:80c::	2804:14c:f287:4239::	TCP	74	443 → 49530 [FIN, ACK] Seq=74 Ack=2 Win=1045 Len=0
245	3.999448	2804:14c:f287:4239::	2800:3f0:4001:80c::	TCP	74	49530 → 443 [ACK] Seq=2 Ack=75 Win=517 Len=0
312	5.509090	2804:14c:f287:4239::	2600:1419:2a00:9::2	TCP	86	56391 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM
313	5.509380	2804:14c:f287:4239::	2600:1419:2a00:9::2	TCP	86	55113 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM
315	5.551968	2600:1419:2a00:9::2	2804:14c:f287:4239::	TCP	86	80 → 56391 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1440 SACK_PERM WS=128
316	5.552104	2804:14c:f287:4239::	2600:1419:2a00:9::2	TCP	74	56391 → 80 [ACK] Seq=1 Ack=1 Win=132352 Len=0
317	5.552242	2600:1419:2a00:9::2	2804:14c:f287:4239::	TCP	86	80 → 55113 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1440 SACK_PERM WS=128
318	5.552360	2804:14c:f287:4239::	2600:1419:2a00:9::2	TCP	74	55113 → 80 [ACK] Seq=1 Ack=1 Win=132352 Len=0
319	5.552459	2804:14c:f287:4239::	2600:1419:2a00:9::2	HTTP	525	GET / HTTP/1.1
320	5.595753	2600:1419:2a00:9::2	2804:14c:f287:4239::	TCP	74	80 → 56391 [ACK] Seq=1 Ack=452 Win=64384 Len=0
321	5.605432	2600:1419:2a00:9::2	2804:14c:f287:4239::	HTTP	768	HTTP/1.1 200 OK (text/html)
323	5.647398	2804:14c:f287:4239::	2600:1419:2a00:9::2	TCP	74	56391 → 80 [ACK] Seq=452 Ack=695 Win=131584 Len=0
326	5.752671	2804:14c:f287:4239::	2600:1419:2a00:9::2	HTTP	469	GET /favicon.ico HTTP/1.1
327	5.839143	2600:1419:2a00:9::2	2804:14c:f287:4239::	TCP	74	80 → 56391 [ACK] Seq=695 Ack=847 Win=64128 Len=0
332	6.952466	2804:14c:f287:4239::	2600:1419:2a00:9::2	HTTP	978	HTTP/1.1 404 Not Found (text/html)
333	7.000847	2804:14c:f287:4239::	2600:1419:2a00:9::2	TCP	74	56391 → 80 [ACK] Seq=847 Ack=1599 Win=132352 Len=0
339	10.435199	192.168.0.205	195.133.244.34	TLSv1.2	108	Application Data

Figura 2: Ciclo de Vida Completo da Conexão TCP: Three-Way Handshake (SYN, SYN-ACK, ACK) e Sequência de Encerramento (FIN, ACK).

3.1 Criação de Conexão TCP

A criação da conexão é realizada através do procedimento conhecido como Three-Way Handshake (aperto de mão de três vias), que garante que ambos os lados estejam prontos para a comunicação e com os parâmetros sincronizados.

→ Processo Observado (Pacotes 312 a 317):

- ◆ SYN (Sincronização - Pacote 312): O cliente (2600:1419...) envia um pacote com a flag SYN para o servidor, indicando a intenção de iniciar uma conexão.
- ◆ SYN-ACK (Sincronização e Confirmação - Pacote 313): O servidor responde com as flags SYN e ACK, sinalizando que reconheceu o pedido e também está pronto para estabelecer a sessão.

- ◆ ACK (Confirmação - Pacote 317): O cliente envia um pacote final com a flag ACK, confirmando o recebimento da resposta. A conexão é oficialmente estabelecida e a troca de dados pode começar.

3.2 Encerramento da Conexão TCP

O encerramento de uma conexão TCP também segue um processo estruturado, garantindo que não haja perda de dados.

→ Processo Observado (Pacotes 323 e 333):

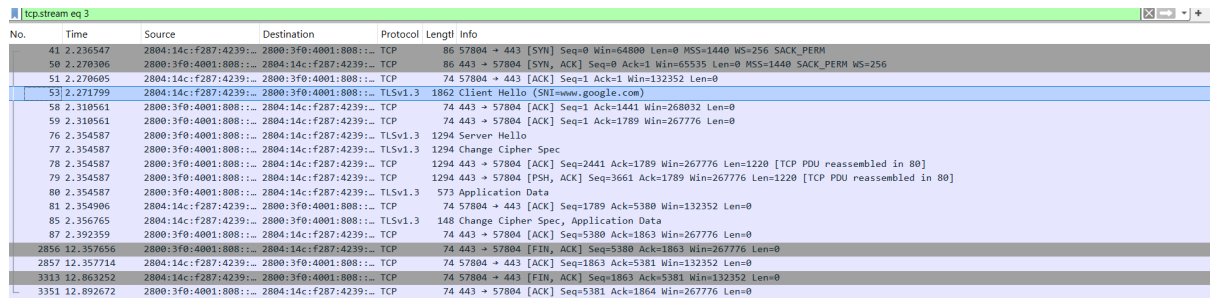
- ◆ FIN (Finish - Pacote 323): Um dos lados (o cliente, no caso 2600:1419...) envia um pacote com a flag FIN (e ACK), indicando a intenção de encerrar sua parte da comunicação.
- ◆ ACK (Confirmação): O outro lado responde com uma confirmação (ACK), reconhecendo o pedido de término.
- ◆ FIN (Finish - Pacote 333): Após o primeiro lado, o segundo lado também envia seu próprio pacote FIN (e ACK), indicando que encerrou sua transmissão.
- ◆ ACK Final: O primeiro lado responde com o ACK final, encerrando completamente a sessão TCP.

3.3 Sobre a Captura de Pacotes

A captura (Figura 2) ilustra de forma clara o ciclo de vida do TCP, desde o estabelecimento até o fechamento.

- Transferência de Dados: Entre as etapas de criação e encerramento, o pacote 319 mostra a requisição GET / HTTP/1.1 enviada pelo cliente, solicitando o conteúdo de uma página web.
- Sequência Completa: A observação da sequência completa de pacotes, que inclui as flags SYN, ACK, FIN, e a requisição HTTP de dados, confirma a operação do TCP como um protocolo de transporte confiável e orientado à conexão, essencial para serviços como a transferência de páginas web.
- Múltiplas Conexões: O screenshot também mostra múltiplas trocas de dados HTTP e TLS (ex: pacote 0), demonstrando que o protocolo TCP opera continuamente no ambiente de rede.

4 HTTPS



The image shows a Wireshark packet capture of a TLS handshake and data transfer over TCP. The packets are numbered 41 to 3351. The handshake sequence includes: Client Hello (53), Server Hello (76), Change Cipher Spec (77), and Application Data (80). The data transfer consists of multiple packets of encrypted data (Application Data) sent from the client to the server.

No.	Time	Source	Destination	Protocol	Length	Info
41	2.236547	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TCP	86	57804 → 443 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM
50	2.270306	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	86	443 → 57804 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1440 SACK_PERM WS=256
51	2.270605	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TCP	74	57804 → 443 [ACK] Seq=1 Ack=1 Win=132352 Len=0
53	2.271799	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TLSv1.3	1862	Client Hello [SNI=www.google.com]
58	2.310561	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	74	443 → 57804 [ACK] Seq=1 Ack=1441 Win=268032 Len=0
59	2.310561	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	74	443 → 57804 [ACK] Seq=1 Ack=1789 Win=267776 Len=0
76	2.354587	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TLSv1.3	1294	Server Hello
77	2.354587	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TLSv1.3	1294	Change Cipher Spec
78	2.354587	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	1294	443 → 57804 [ACK] Seq=2441 Ack=1789 Win=267776 Len=1220 [TCP PDU reassembled in 80]
79	2.354587	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	1294	443 → 57804 [PSH, ACK] Seq=3661 Ack=1789 Win=267776 Len=1220 [TCP PDU reassembled in 80]
80	2.354587	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TLSv1.3	573	Application Data
81	2.354906	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TCP	74	57804 → 443 [ACK] Seq=1789 Ack=5380 Win=132352 Len=0
85	2.356765	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TLSv1.3	148	Change Cipher Spec, Application Data
87	2.392359	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	74	443 → 57804 [ACK] Seq=5380 Ack=1863 Win=267776 Len=0
2856	12.357656	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	74	443 → 57804 [FIN, ACK] Seq=5380 Ack=1863 Win=267776 Len=0
2857	12.357714	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TCP	74	57804 → 443 [ACK] Seq=1863 Ack=5381 Win=132352 Len=0
3313	12.863252	2804:14c:f287:4239::...	2800:3f0:4001:808::...	TCP	74	57804 → 443 [FIN, ACK] Seq=1863 Ack=5381 Win=132352 Len=0
3351	12.892672	2800:3f0:4001:808::...	2804:14c:f287:4239::...	TCP	74	443 → 57804 [ACK] Seq=5381 Ack=1864 Win=267776 Len=0

Figura 3: Handshake TLS (Client Hello/Server Hello) e Fluxo de Dados Criptografados (Application Data) em Conexão TCP.

O HTTPS é o protocolo utilizado para comunicação segura na web, combinando o HTTP com o protocolo TLS (Transport Layer Security). O TLS é responsável por estabelecer a criptografia e garantir a autenticidade da comunicação. Sua captura utiliza a Camada de Transporte confiável do TCP para suportar a segurança do TLS.

Estabelecimento da Conexão TLS (Handshake)

Antes de qualquer transferência de dados segura, o TLS realiza um processo de Handshake criptografado, que tem como objetivo autenticar as partes e negociar as chaves de criptografia.

- ➔ Client Hello (Pacote 53): O processo se inicia quando o cliente envia a mensagem Client Hello ao servidor, listando os algoritmos de criptografia e versões TLS que suporta.
- ➔ Server Hello (Pacote 76): O servidor responde com a mensagem Server Hello, selecionando os parâmetros e enviando seu Certificado digital (para autenticação).
- ➔ Troca de Chaves (Pacote 77): O cliente finaliza o Handshake com a mensagem Change Cipher Spec, utilizando a chave negociada para ativar a criptografia e iniciar a sessão segura.

Transferência de Dados Criptografados

Após o Handshake, a comunicação de dados ocorre de forma segura:

- Os pacotes de dados do aplicativo são classificados como Application Data (por exemplo, pacote 79), indicando que o conteúdo real do HTTP está criptografado e ilegível para qualquer agente externo que monitore a rede, incluindo o Wireshark.
- Essa criptografia protege informações sensíveis, como credenciais e dados pessoais.

Encerramento da Conexão

O encerramento da sessão segura é visto no nível do TCP. O processo é sinalizado pelos pacotes FIN, ACK (pacotes 2856 e 3312), garantindo que os recursos do TCP sejam liberados após o término da sessão TLS.

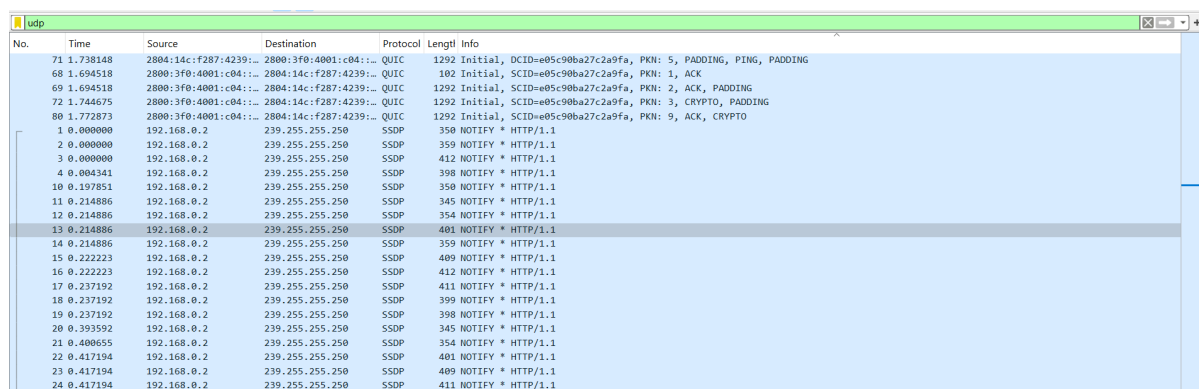
Observação sobre o Filtro de Captura:

Para obter a Figura 3, foi utilizado o recurso "Follow TCP Stream" do Wireshark, o que gerou um filtro específico (ex: tcp.stream eq 3). Este método foi escolhido propositalmente em vez de usar apenas o filtro tls para demonstrar a relação hierárquica entre os protocolos.

A captura mostra que o TLS (que cuida da criptografia com mensagens como Client Hello e Server Hello) depende do serviço confiável do TCP para funcionar. O filtro tcp.stream isola o ciclo de vida completo de uma única sessão segura, mostrando tanto a criação e o encerramento da conexão TCP quanto o Handshake TLS e o Application Data.

5 UDP

O UDP (User Datagram Protocol) é um protocolo de transporte minimalista, projetado para ser rápido e sem complicações. Ao contrário do TCP, ele não estabelece conexões nem garante a entrega dos dados. Essa abordagem é ideal para aplicações em tempo real, como chamadas de vídeo e streaming, onde uma pequena perda de pacotes é preferível a atrasos causados por retransmissões. Por não verificar se os dados chegaram ao destino, o UDP é extremamente eficiente, com cabeçalhos menores e latência reduzida. No entanto, a aplicação precisa lidar sozinha com eventuais perdas ou desordem nos pacotes. Além disso, a falta de handshake o torna vulnerável a ataques DDoS.



No.	Time	Source	Destination	Protocol	Length	Info
71	1.738148	2880:14c:f287:4239::...	2880:3f0:4001:c04::...	QUIC	1292	Initial, DCID=e05c90ba27c2a9fa, PKN: 5, PADDING, PING, PADDING
68	1.694518	2880:3f0:4001:c04::...	2880:14c:f287:4239::...	QUIC	102	Initial, SCID=e05c90ba27c2a9fa, PKN: 1, ACK
69	1.694518	2880:3f0:4001:c04::...	2880:14c:f287:4239::...	QUIC	1292	Initial, SCID=e05c90ba27c2a9fa, PKN: 2, ACK, PADDING
72	1.744675	2880:3f0:4001:c04::...	2880:14c:f287:4239::...	QUIC	1292	Initial, SCID=e05c90ba27c2a9fa, PKN: 3, CRYPTO, PADDING
80	1.772873	2880:3f0:4001:c04::...	2880:14c:f287:4239::...	QUIC	1292	Initial, SCID=e05c90ba27c2a9fa, PKN: 9, ACK, CRYPTO
1	0.000000	192.168.0.2	239.255.255.250	SSDP	350	NOTIFY * HTTP/1.1
2	0.000000	192.168.0.2	239.255.255.250	SSDP	412	NOTIFY * HTTP/1.1
3	0.000000	192.168.0.2	239.255.255.250	SSDP	398	NOTIFY * HTTP/1.1
4	0.004341	192.168.0.2	239.255.255.250	SSDP	350	NOTIFY * HTTP/1.1
10	0.197851	192.168.0.2	239.255.255.250	SSDP	345	NOTIFY * HTTP/1.1
11	0.214886	192.168.0.2	239.255.255.250	SSDP	354	NOTIFY * HTTP/1.1
12	0.214886	192.168.0.2	239.255.255.250	SSDP	401	NOTIFY * HTTP/1.1
13	0.214886	192.168.0.2	239.255.255.250	SSDP	359	NOTIFY * HTTP/1.1
14	0.214886	192.168.0.2	239.255.255.250	SSDP	409	NOTIFY * HTTP/1.1
15	0.222223	192.168.0.2	239.255.255.250	SSDP	412	NOTIFY * HTTP/1.1
16	0.222223	192.168.0.2	239.255.255.250	SSDP	411	NOTIFY * HTTP/1.1
17	0.237192	192.168.0.2	239.255.255.250	SSDP	399	NOTIFY * HTTP/1.1
18	0.237192	192.168.0.2	239.255.255.250	SSDP	398	NOTIFY * HTTP/1.1
19	0.237192	192.168.0.2	239.255.255.250	SSDP	345	NOTIFY * HTTP/1.1
20	0.393592	192.168.0.2	239.255.255.250	SSDP	354	NOTIFY * HTTP/1.1
21	0.400655	192.168.0.2	239.255.255.250	SSDP	401	NOTIFY * HTTP/1.1
22	0.417194	192.168.0.2	239.255.255.250	SSDP	409	NOTIFY * HTTP/1.1
23	0.417194	192.168.0.2	239.255.255.250	SSDP	411	NOTIFY * HTTP/1.1
24	0.417194	192.168.0.2	239.255.255.250	SSDP	411	NOTIFY * HTTP/1.1

Figura 4: Tráfego UDP e o Protocolo SSDP (Multicast) – Exemplo de Comunicação Não Confiável e Sem Conexão.

A Figura 4 ilustra o uso do UDP na rede local, confirmando as características teóricas do protocolo. Ao filtrar a captura por UDP, observou-se principalmente a ocorrência do tráfego SSDP (Simple Service Discovery Protocol), que é um exemplo clássico de serviço que utiliza este protocolo.

Características Observadas:

- ➔ **Natureza Sem Conexão:** A principal evidência é a ausência total dos pacotes de controle SYN e ACK na camada de transporte. As transações (pacotes com a informação NOTIFY * HTTP/1.1) ocorrem de forma autônoma e discreta, sem a sobrecarga de um Handshake prévio.
- ➔ **Uso de Multicast:** Muitos pacotes SSDP (por exemplo, pacotes 1 a 24) são enviados para o endereço de destino 239.255.255.250. Esse é um endereço IP de multicast, e o UDP é o protocolo de transporte obrigatório para este tipo de comunicação, onde uma informação é enviada a um grupo de dispositivos, sem exigir confirmação individual de recebimento.
- ➔ **Eficiência:** O tamanho dos pacotes é relativamente pequeno (cerca de 350 a 481 bytes), refletindo os cabeçalhos menores e a latência reduzida do UDP.

A presença do tráfego SSDP demonstra que o UDP é o protocolo ideal para a descoberta de serviços na rede, onde a rapidez e a capacidade de atingir vários receptores simultaneamente (multicast) são mais importantes do que garantir que cada notificação individual chegue ao destino.

6 Conclusão

Este trabalho prático permitiu analisar na prática o funcionamento dos principais protocolos de rede, aplicando os conceitos da disciplina por meio da ferramenta Wireshark.

- ➔ **Comparação dos Protocolos de Transporte:**
 - ◆ O TCP demonstrou seu papel fundamental ao estabelecer conexões confiáveis por meio de seu handshake de três vias (SYN, SYN-ACK, ACK), garantindo a entrega e a ordem dos dados.
 - ◆ Em claro contraste, o UDP prioriza a velocidade e a baixa latência com seu formato mais simples, operando de forma sem conexão e sem mecanismos de garantia, ideal para serviços sensíveis ao tempo.
- ➔ **Segurança e Endereçamento:**
 - ◆ O protocolo ARP confirmou seu papel fundamental no mapeamento de endereços lógicos (IP) para endereços físicos (MAC) dentro da rede local.
 - ◆ A análise do HTTPS/TLS ilustrou como o TCP é utilizado como camada de transporte para o estabelecimento de uma comunicação criptografada, destacando a importância do Handshake TLS para a negociação de chaves e a proteção dos dados via Application Data.

→ Eficiência da Ferramenta:

- ◆ A ferramenta Wireshark se mostrou eficiente para capturar e filtrar pacotes, facilitando a compreensão do tráfego de rede. Foi possível identificar padrões de comunicação, desde a abertura de conexões até a transferência de dados e seu encerramento.

Os resultados demonstraram claramente as diferenças estruturais entre os protocolos e seus casos de uso típicos, consolidando a compreensão de como o tráfego de dados é gerenciado nas redes.

7 Bibliografia

- TANENBAUM, Andrew S.; WETHERALL, David J. Redes de Computadores. 5. ed. São Paulo: Pearson, 2011.
- FOROUZAN, Behrouz A. Data Communications and Networking. 5th ed. McGraw-Hill, 2012.
- SANDERS, Chris. Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems. 3rd ed. No Starch Press, 2017.